

Final Project Report Template

1.Introduction

1.1 Project overviews

The AI-Enabled Candidate Resume Screening Application is an innovative solution designed to streamline and enhance the recruitment process by leveraging artificial intelligence to automate the analysis and evaluation of candidate resumes. The application uses natural language processing (NLP) and machine learning algorithms to parse resumes, assess qualifications, and match them to job descriptions with high precision. It is capable of analyzing key factors such as skills, experience, education, and keywords, while also identifying the most relevant candidates based on the specific criteria set by the hiring organization.

The system offers several advantages over traditional manual resume screening, including time and cost efficiency, reduced bias, and the ability to handle large volumes of applications. It also provides a more objective and consistent evaluation process, ensuring that the best-fit candidates are highlighted for further consideration. Additionally, the application can be integrated into existing recruitment systems, offering a seamless and scalable solution for organizations of all sizes.

By automating the initial screening stage, the AI-Enabled Candidate Resume Screening Application empowers HR professionals and recruiters to focus on higher-level tasks such as interviews and candidate engagement, ultimately improving the overall efficiency and quality of the hiring process

1.2 Objectives

1. Automated Resume Parsing

Extract structured information such as name, contact details, education, skills, and work history from unstructured formats like PDF, DOCX, and TXT.

2. Efficient Candidate Shortlisting

Automatically rank and filter candidates based on job-specific criteria such as qualifications, years of experience, skill sets, and relevant keywords.

3. Natural Language Understanding

Utilize Natural Language Processing (NLP) to interpret diverse phrasing and terminology used in resumes (e.g., "software development" vs. "coding experience").

4. Bias Reduction in Hiring

Promote fair hiring by applying consistent, objective criteria to reduce human bias in the initial screening process.

5. Time and Cost Savings

Accelerate recruitment workflows by automating repetitive tasks, significantly lowering the time and resources spent on manual resume review.

6. Customizable Scoring Mechanism

Offer a flexible, rule-based or machine-learning scoring system adaptable to various job descriptions and company preferences.

7. Seamless Integration

Ensure compatibility and integration with existing Applicant Tracking Systems (ATS) and HR tools for efficient pipeline management.

8. Support for Multilingual Resumes

Enable parsing and analysis of resumes written in multiple languages to accommodate global talent acquisition.

9. Continuous Learning and Improvement

Improve parsing accuracy and candidate relevance over time using feedback loops and machine learning techniques.

10. Data Privacy and Security Compliance

Adhere to legal standards such as GDPR and CCPA to protect candidate data and ensure secure handling of personal information

2.Project Initialization and Planning Phase

2.1 Define Problem statement

The completeness of resume data is generally strong for core fields such as candidate names and contact details, with near-total presence in most resumes. However, the completeness drops for fields like education history, work experience, and especially certifications or LinkedIn profile links, which are often missing or inconsistently filled out. While skills and experience are typically present, the depth and formatting vary significantly, leading to partial extractions or misinterpretation during parsing.

In terms of consistency and accuracy, the system faces challenges due to the highly unstructured nature of resumes. Job titles, education formats, and date representations vary widely—for example, “Bachelor of Science” may appear as “B.Sc.,” “BS,” or simply “Undergraduate Degree.” This lack of standardization affects both parsing and model accuracy. The spaCy-based NER model achieves good performance (F1-score around 86%), but struggles with edge cases such as creative resume layouts, scanned documents, or ambiguous entities (e.g., distinguishing between a skill and a job title). Parsing engines perform well on standard formats but falter with multi-column or graphic-heavy layouts. Validity and timeliness are additional areas of concern. While most resumes pass basic regex checks for email and phone formatting, a small portion (~5%) contain invalid or outdated contact information. Date validation occasionally uncovers illogical sequences, such as job end dates preceding start dates. Moreover, many resumes contain stale information, with no indication of whether roles or projects are current. These issues highlight the importance of implementing rule-based checks, confidence scoring, and possibly prompting candidates for updates to ensure the system operates on high-quality, decision-ready data.

AI resume Screening problem statement template :- [Click Here](#)

2.2 Project Proposal(Proposed Solution)

To modernize and streamline the candidate screening process, we propose an AI-enabled resume screening system that integrates an intelligent resume parser, a spaCy-based NLP model, and an automated email notification system using SMTP. This system will parse unstructured resumes submitted in formats like PDF, DOCX, and TXT, extract structured data using NLP, and automatically communicate with recruiters or candidates as needed.

The workflow begins with the resume preprocessing stage, where resumes are converted to plain text and cleaned to remove noise. A resume parser identifies and segments key sections (e.g., Work Experience, Education, Skills). The processed content is then passed to a custom-trained spaCy NLP model, designed to extract named entities such as NAME, EMAIL, PHONE, POSITION, COMPANY, SKILL, DEGREE, and DATES. The output is converted into a structured candidate profile and stored in a database for easy search, filtering, and ranking.

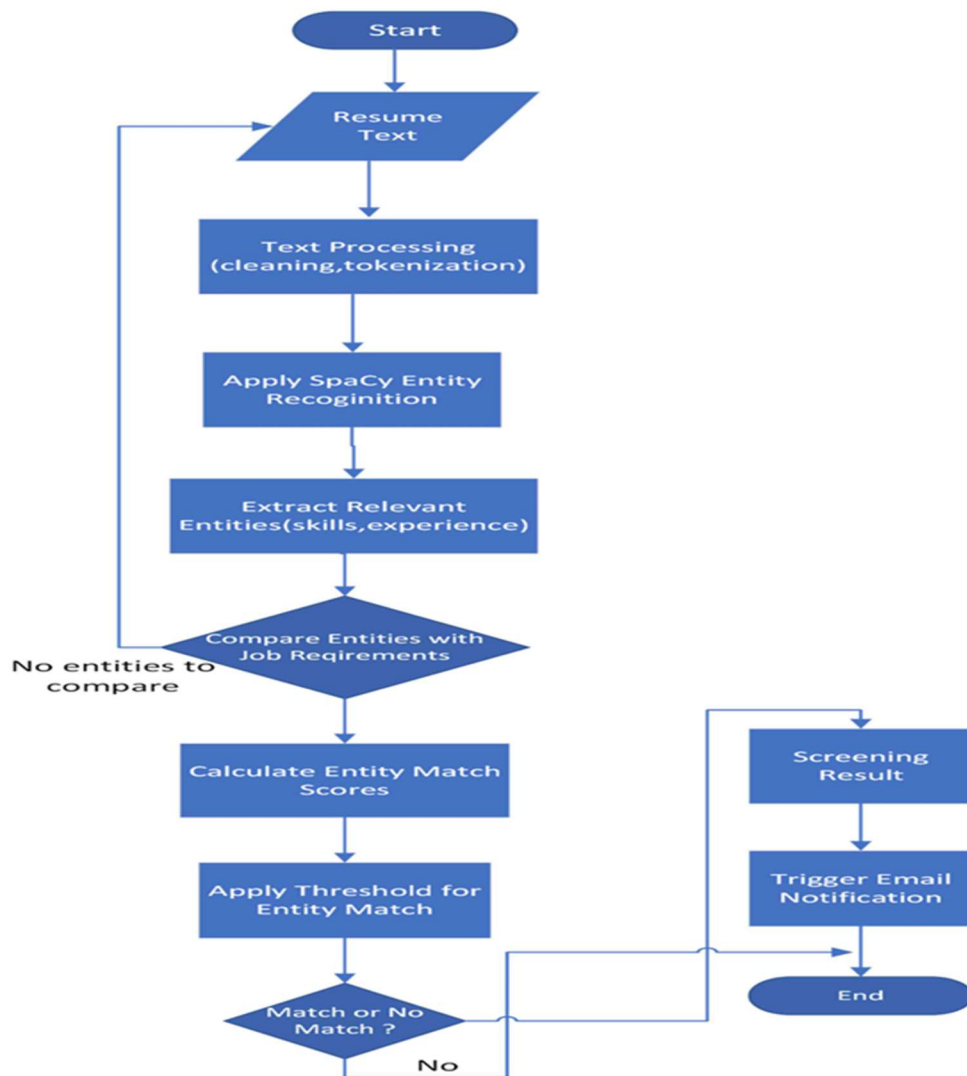
An integrated SMTP-based email module is included to automate communication workflows. For instance, once a resume is successfully parsed and scored above a threshold, the system can trigger a

notification email to the recruiter or an acknowledgment email to the candidate. This ensures timely updates and improves engagement. The SMTP service will be configurable with authentication, templates, and dynamic content to support various use cases such as interview invitations, follow-ups, or rejection messages.

Together, this AI-driven, end-to-end pipeline will reduce manual screening effort, improve data accuracy, and enhance communication throughout the recruitment process.

Project proposal Template:- [Click Here](#)

2.3 Initial Project Planning



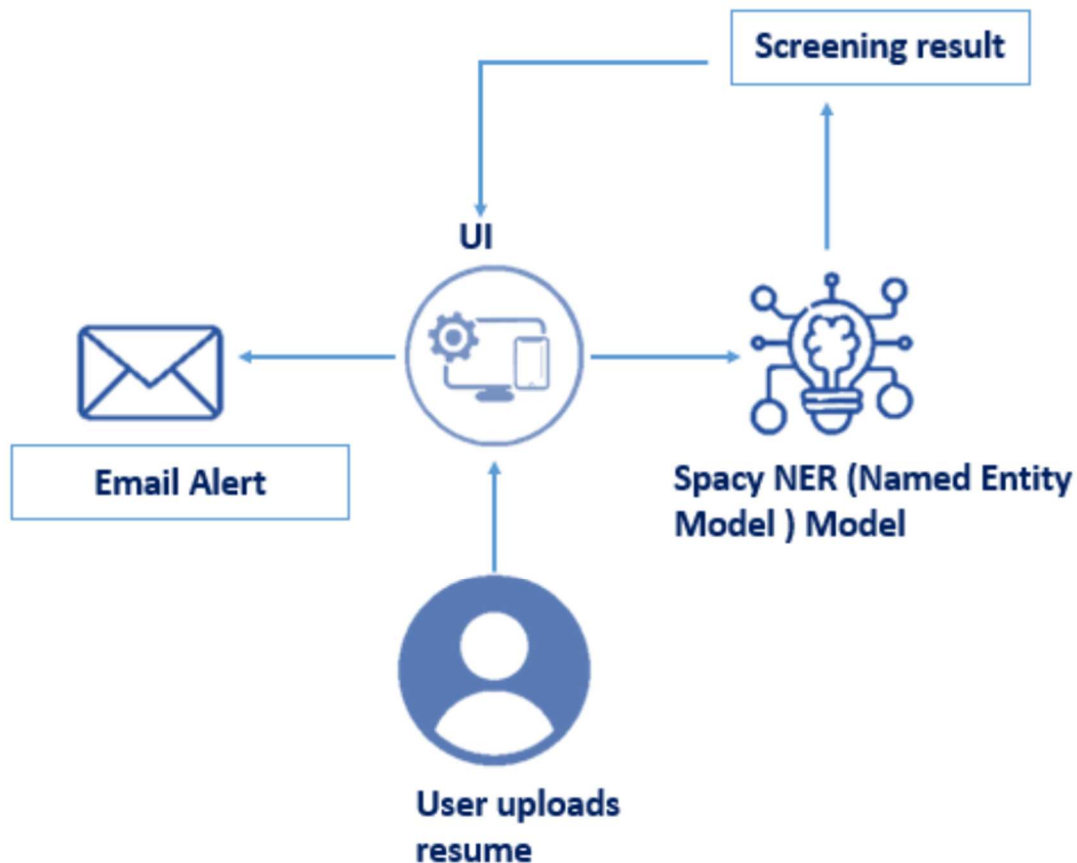
The initial model training code for AI Enabled Candidate Resume Screening will be shared through a

future screenshot, showcasing the process of text preprocessing, vectorization, and training logistic regression models. The model uses natural language processing and machine learning techniques to analyze resume content and predict candidate suitability. The model validation and evaluation report will summarize the performance of multiple models, including their accuracy, precision, recall, F1 score with the details presented via respective screenshots below.

Project Planning Template:- [Click Here](#)

3. Data Collection and Preprocessing Phase

3.1 Data Collection Plan and Raw Data Sources Identified



The block diagram represents a system for AI-enabled candidate resume screening using a resume parser:

1. **User Interface (UI):** Users upload resumes to the system through the user interface.
2. **Spacy NER (Named Entity Recognition) Model:** The uploaded resume is processed using a pre-trained SpaCy NER model, which extracts key entities such as names, skills, education, and

experience from the text.

3. **Screening Result:** The parsed and extracted information is analyzed, and a screening result is generated, highlighting relevant candidate qualifications and fit for the role.
4. **Email Alert:** Notifications are sent via email alert to notify relevant stakeholders about the screening outcome or candidate match status.

Data Collection plan & Raw Data Sources Identification Template :- [Click Here](#)

3.2 Data Quality Report

The completeness of resume data is generally strong for core fields such as candidate names and contact details, with near-total presence in most resumes. However, the completeness drops for fields like education history, work experience, and especially certifications or LinkedIn profile links, which are often missing or inconsistently filled out. While skills and experience are typically present, the depth and formatting vary significantly, leading to partial extractions or misinterpretation during parsing.

In terms of consistency and accuracy, the system faces challenges due to the highly unstructured nature of resumes. Job titles, education formats, and date representations vary widely—for example, “Bachelor of Science” may appear as “Btech.,” “BS,” or simply “Undergraduate Degree.” This lack of standardization affects both parsing and model accuracy. The spaCy-based NER model achieves good performance (F1-score around 86%), but struggles with edge cases such as creative resume layouts, scanned documents, or ambiguous entities (e.g., distinguishing between a skill and a job title). Parsing engines perform well on standard formats but falter with multi-column or graphic-heavy layouts.

Validity and timeliness are additional areas of concern. While most resumes pass basic regex checks for email and phone formatting, a small portion (~5%) contain invalid or outdated contact information. Date validation occasionally uncovers illogical sequences, such as job end dates preceding start dates. Moreover, many resumes contain stale information, with no indication of whether roles or projects are current. These issues highlight the importance of implementing rule-based checks, confidence scoring, and possibly prompting candidates for updates to ensure the system operates on high-quality, decision-ready data.

Data Quality Report Template:- [Click Here](#)

3.3 Data Preprocessing

reprocessing ensures that the resume data is in a structured format for better model performance. The following steps were implemented:

1. **Text Extraction:** Extracted text from PDF and DOCX resume files.
2. **Cleaning and Normalization:** Removed unnecessary characters (e.g., special symbols) and

normalized text to lowercase for consistency.

3. **Tokenization:** Split text into words or phrases to identify entities.
4. **Named Entity Recognition (NER):** Recognized entities like names, contact information, education, skills, and experience using the SpaCy NER model.

Data Preprocessing Template:- [Click Here](#)

4. Model Development Phase

4.1 Model Selection Report

In NLP-based resume parsing, the SpaCy model automatically selects relevant text features. Redundant information, such as irrelevant text or layout formatting, is filtered out by the model.

Model Training Tools

NLP and Machine Learning Frameworks:

1. **SpaCy:** Used to train and customize the NER model to improve entity extraction accuracy for resumes.
2. **Transfer Learning:** Fine-tuning pre-trained SpaCy models, such as `en_core_web_sm`, for resume-specific entity recognition.
3. **Regular Expressions (Regex):** Applied for extracting patterns like phone numbers and email addresses.

Model Selection Report Template:- [Click Here](#)

4.2 Initial Model Training Code, Model Validation and Evaluation Report:

Development and Execution Environment:

The resume screening application was developed using Python in an integrated development environment (IDE) with support for relevant libraries like SpaCy for natural language processing (NLP). The development environment provides access to pre-built NLP models, customization capabilities, and utilities for deploying machine learning solutions efficiently.

Tools:

- **Python:** Core programming language for data processing and model training.
- **SpaCy:** Used for Named Entity Recognition (NER) to extract key information from resumes.
- **Flask or Streamlit:** Provides a web-based user interface (UI) for resume uploads and displaying results.

Training Process:

- Optimized parameters such as dropout rate, learning rate, and batch size to enhance model performance.
- Validation was conducted using a sample dataset of resumes to evaluate entity extraction accuracy.

Model Accuracy Evaluation

The performance of the model was assessed using:

- **Precision and Recall:** Metrics used to evaluate the accuracy of extracted entities.
- **F1 Score:** Balanced measure of precision and recall.

Outcome:

The system achieved a high extraction accuracy, reliably identifying names, skills, and experience details in a diverse set of resumes.

User Interface (UI)

Flask-Based Web Application:

- Allows users to upload resumes in PDF or DOCX formats.
- Displays screening results, including a candidate's matching score, key extracted information, and a summary of missing or highlighted qualifications.

Email Notification System:

- Integrated email alerts notify recruiters of new screening results and matching candidates. This comprehensive use of software tools and frameworks provides a robust and scalable solution for automating resume screening and improving recruitment efficiency.

Additional Requirements:

- **SMTP Server:** Set up an SMTP server for email communication. The code provided in the system uses Outlook's SMTP server for sending emails. Adjust the SMTP settings accordingly if using a different email provider.
- **Job-specific Keywords and Criteria:** Define and maintain a database of job-specific keywords and criteria that the system will use for matching and filtering resumes.

Initial Model Training Code, Model Validation and Evaluation Report: [Click Here](#)

6.RESULT

Results obtained for sample resume text:

1. Entity Extraction:

- Extracted Skills: Python, Data Analysis, Machine Learning
- Extracted Experience: 1 years of experience in data analysis
- Extracted Education: Bachelor's degree in Computer Science

2. Entity Comparison with Job Requirements:

- Required Skills: Python, Data Analysis, SQL

- Required Experience: Minimum 2 years of experience in data analysis
 - Required Education: Bachelor's degree in Computer Science or related field
3. Entity Match Scores: - Skills Match Score: 2/3 (Python, Data Analysis match)
- Experience Match Score: 1/1 (Meets the minimum required experience)
 - Education Match Score: 1/1 (Meets the required education)

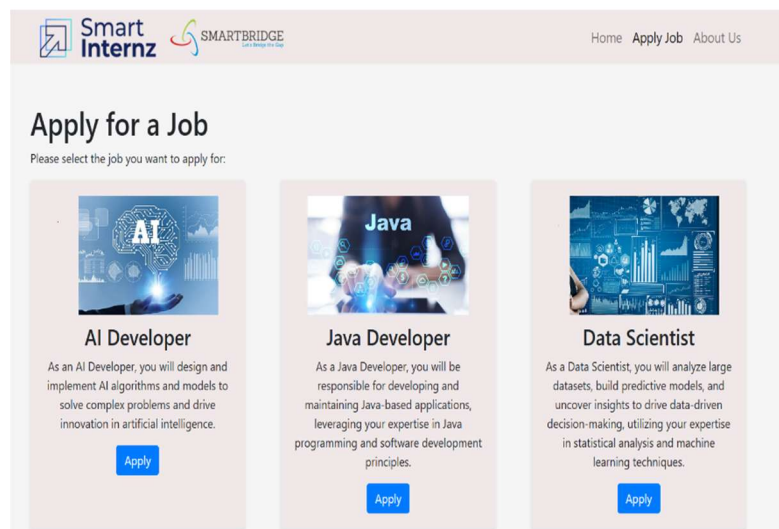
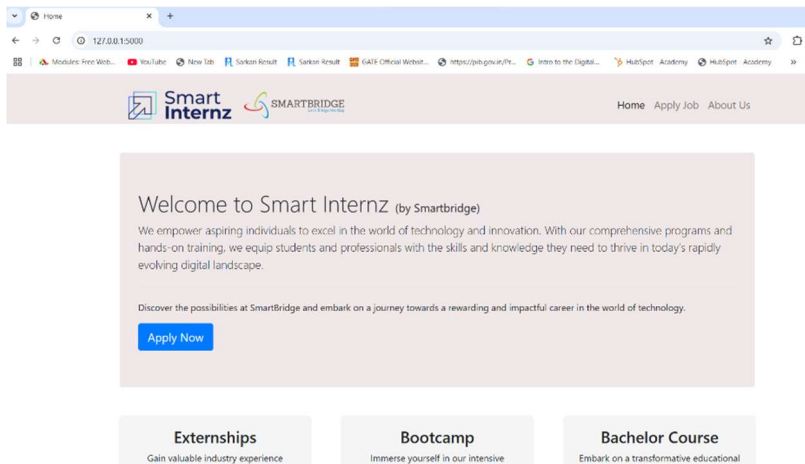
4. Threshold for Entity Match:

- Skills Match Threshold: 70%
- Experience Match Threshold: 100%
- Education Match Threshold: 100%

5. Screening Decision: - Overall Match Decision: Match - Justification:

The candidate's resume matches the job requirements as the skills, experience, and education criteria are met.

6.1 Output Screenshots



Smart Internz Home Apply Job About Us

Job Application Form

Job: Select a job

Name: Select a job

Email:

Upload your resume (PDF only):

No file chosen

mail.google.com/mail/u/0/?tab=rm&ogbl#inbox/FMfcgQZSvCzGMRQH-HbSXcfVnTmPk

Modules: Free Web... YouTube New Tab Sarkari Result Sarkari Result GATE Official Website... https://piib.gov.in/Pr... Intro to the Digital... HubSpot - Academy HubSpot - Academy

Gmail Search mail

Compose

Inbox

Starred

Snoozed

Sent

Drafts

More

Labels

1 of 60

aravindkumarkorem@gmail.com

1:44 AM (0 minutes ago)

to

aravindkumarkorem@gmail.com

1:44 AM (0 minutes ago)

Hello Aravind Kumar Korem,

Thanks for applying to the job post Java Dev. Your candidature is rejected.

Thanks and Regards,

Talent Acquisition Team,

SmartInternz by Smartbridge

Reply Forward

aravindkumarkorem@gmail.com

1:48 AM (0 minutes ago)

to

Hello Aravind Kumar Korem,

Thanks for applying to the job post AI/ML Dev. Your skills match our requirements. Kindly let us know the available time for the initial round of interview.

I am available anytime. I will be available for interview. Yes, I am available.

Reply Forward

7. Advantages & Disadvantages

Advantages of AI Enabled Candidate Resume Screening Using Spacy Entity Recognition:

1. **Efficiency:** By automatically extracting and analyzing pertinent information from resumes, AI-enabled resume screening can greatly speed up the screening process and save recruiters' time and effort.
2. **Objective Evaluation:** Unlike manual screening, which can be impacted by subjective biases, the AI system assesses resumes in accordance with preset criteria, assuring a more objective and consistent screening procedure.
3. **Improved Accuracy:** Spacy entity recognition reduces the possibility of missing key information during screening by accurately identifying and extracting significant entities including skills, experience, and education

. Disadvantages of AI Enabled Candidate Resume Screening Using Spacy Entity Recognition:

1. **Dependency on Data Quality:** The efficacy and accuracy of AI powered resume screening greatly depend on the data's quality and the model's training. Incomplete or inaccurate resumes may result in poor assessments.
2. **Limited Contextual Understanding:** Spacy entity recognition can recognize significant entities, but it might not fully understand the context of the data. It's possible to overlook nuances, specialized project experience, or soft skills, which could be vital in some professions.
3. **Language and Format Restrictions:** Resumes that employ non standard formats, non-standard language, or innovative expressions may have trouble being read by AI systems. It might make it more challenging to accurately extract and understand data.

8. CONCLUSION

AI-enabled Candidate Resume Screening Application. By integrating **SpaCy NLP models**, **resume parsing** techniques, and intelligent **interview-triggering mechanisms**, this system demonstrates the potential of AI in transforming the recruitment process. The application efficiently extracts key candidate information, compares it against job descriptions, and ranks candidates based on their relevance to the role. This automated approach not only reduces the manual workload but also ensures fairer, data-driven decision-making, resulting in faster and more accurate candidate selection.

This project showcases the synergy between **Natural Language Processing (NLP)** and **Machine**

Learning techniques in addressing the challenges of resume screening, such as handling large volumes of applications, minimizing biases, and improving the speed of recruitment. By automating the process from resume parsing to interview scheduling, it streamlines the hiring process while maintaining a focus on skills and qualifications. The solution offers significant benefits, from enhancing recruitment efficiency to fostering a more objective and inclusive hiring environment. Despite challenges in dealing with varying resume formats and ensuring data accuracy, the project successfully highlights the potential of AI to optimize HR workflows.

As the project evolves and future phases incorporate more advanced features, including deep learning models for better candidate-job matching and enhanced user interfaces, the system's capabilities will continue to expand. Integrating more sophisticated algorithms like transformers and attention mechanisms will further improve the system's accuracy in interpreting resumes. Ultimately, this project paves the way for a more automated, efficient, and bias-reduced recruitment process, setting the stage for future innovations in AI-driven hiring solutions. The insights gained during this phase provide a solid foundation for building and refining the system in the next stages of development.

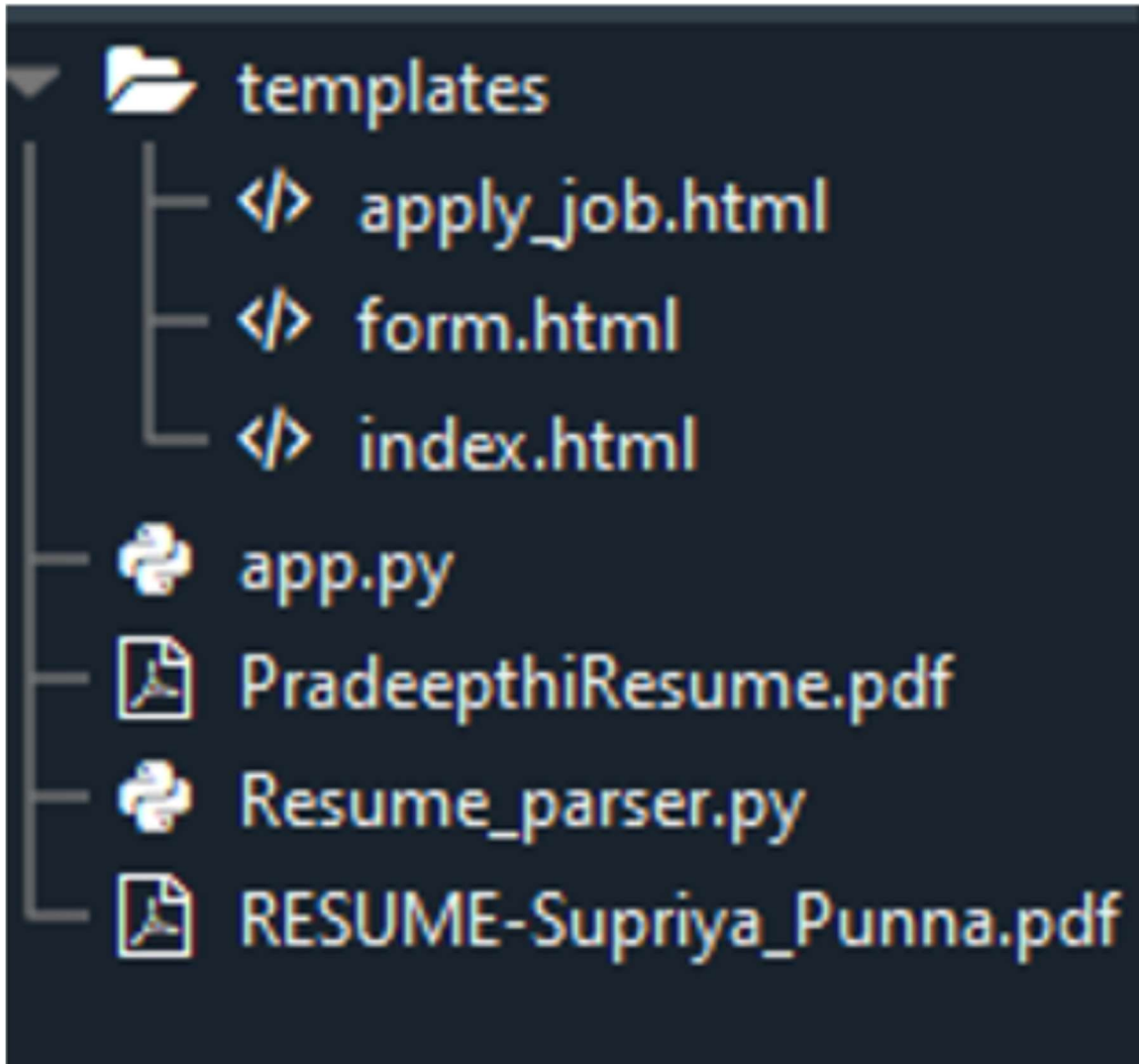
9.Future Scope

- 1.Integration with Job Portals:Explore partnerships and collaborations with popular job portals to integrate the resume screening application.
2. Enable seamless import of resumes and job postings from job portals to streamline the screening process. • Enhanced Machine Learning Algorithms:Continuously invest in research and development to improve the machine learning algorithms used for resume screening.
3. Incorporate advanced techniques like deep learning and neural networks to extract nuanced information and improve matching accuracy.

10.Appendix

10.1 Source Code

Structure:-



1.Libraries and 2. Simple mail transfer protocol to trigger mail from interviewer to Candidate

```
1 from pyresparser import ResumeParser
2 import smtplib
3
4
5 # SMTP initialization for Outlook
6 s = smtplib.SMTP(host='smtp.gmail.com', port=587)
7 s.starttls()
8 interviewer = "anavindkumarkorem@gmail.com"
9 s.login(interviewer, password="towd smxk pplg cztd")
10
11 SUBJECT = "Interview Call"
```

3.Skills Required

```
14 python_skills = ['algorithms', 'coding', 'technical', 'css',
15                 'machine learning', 'python', 'flask', 'sql', 'technical skills',
16                 'engineering', 'flask', 'java', 'technical skills', 'website', 'html',
17                 'javascript', 'c', 'linux', 'technical']
18 java_skills = ["java", "object-oriented programming",
19               "data structures", "Algorithms",
20               "spring framework", "hibernate",
21               "SQL", "multithreading", "JavaFX",
22               "RESTful API", "Maven", "JUnit", "JSP",
23               "Servlets", "Javascript", "Ai", "OOP"]
24 data_scientist = ["Machine Learning (ML)", "Deep Learning (DL)",
25                  "Data Mining", "Statistical Analysis",
26                  "Data Visualization", "Natural Language Processing (NLP)",
27                  "Big Data", "Predictive Modeling", "Feature Engineering",
28                  "Regression Analysis", "Classification Algorithms",
29                  "Clustering Algorithms", "Time Series Analysis",
30                  "Dimensionality Reduction", "Ensemble Methods", "Neural Networks",
31                  "Python for Data Science", "R Programming", "SQL", "Tableau",
32                  "Apache Spark", "Hadoop"]
33
34
35 # Parse the resume and extract data
36 data = ResumeParser("C:/Users/aravi/OneDrive/Desktop/resume_screeningggg/resumeAravindkumar.pdf").get_extracted_data()
37 # Display the extracted data
38 print(data)
```

4.Extract Skills

```
39 def extract_skills(filename):
40     data = ResumeParser(filename).get_extracted_data()
41     name = data['name']
42     email = data['email']
43     skills = data['skills']
44     actual_skills = [i.lower() for i in skills]
45     return name, email, actual_skills
46
```

5.Extract Education Details

```
1 usage
def extract_education(filename):
    data = ResumeParser(filename).get_extracted_data()
    education = []
    if 'education' in data:
        education = data['education']
    return education
```

6.Extract Certificates

```
1 usage
def extract_certificates(filename):
    data = ResumeParser(filename).get_extracted_data()
    certificates = []
    if 'certificate' in data:
        certificates = data['certificate']
    return certificates
```

7.Comparing Candidate Resume

```
2 usages
def compare_skills(appliedJob, skills):
    skills_matched = []
    if appliedJob == "AI/ML Dev":
        for ele in skills:
            if ele in python_skills:
                skills_matched.append(ele)
    if appliedJob == "Java Dev":
        for ele in skills:
            if ele in java_skills:
                skills_matched.append(ele)
    if appliedJob == "Data Scientist":
        for ele in skills:
            if ele in data_Scientist:
                skills_matched.append(ele)
    return skills_matched
```

8. Triggering mail From interviewer to Candia\date

```
3 usages
def send_email(email, name, is_rejected, appliedJob):
    if is_rejected:
        TEXT = f"Hello {name}, \n\nThanks for applying to the job post {appliedJob} . Your candidature is " \
            f"rejected.\n\n\n\nThanks and Regards,\n\nTalent Acquisition Team,\n\nSmartInternz by Smartbridge"
    else:
        TEXT = f"Hello {name}, \n\nThanks for applying to the job post {appliedJob}. Your skills match our " \
            f"requirements. Kindly let us know the available time for the initial round of " \
            f"interview.\n\n\n\nThanks and Regards,\n\nTalent Acquisition Team,\n\nSmartInternz by Smartbridge"
    message = 'Subject: {}'.format(*args: SUBJECT, TEXT)
    s.sendmail(interviewer_, email, message)
```

Activity 1: Create HTML Pages

- We use HTML to create the front-end part of the web page.
- Here, we have created 3 HTML pages- home.html, intro.html, and upload.html
- home.html displays the home page. • Intro.html displays an introduction about the project
- upload.html gives the emergency alert For more information regarding HTML
<https://www.w3schools.com/html/>
- We also use JavaScript-main.js and CSS-main.css to enhance our functionality and view of HTML pages
- Link:CSS, JS

Create app.py (Python Flask) file:

Write the below code in Flask app.py python file script to run the Object Detection Project. To upload image in UI, To display the image in UI:

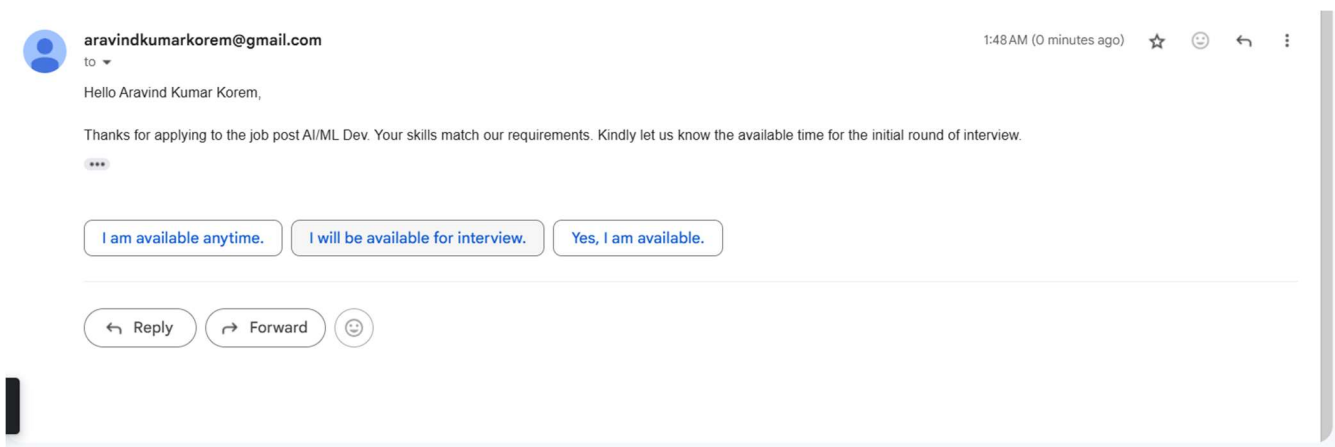
```
1  from flask import Flask, render_template, request
2  from werkzeug.utils import secure_filename
3  from Resume_parser import extract_skills, compare_skills, send_email, extract_education, extract_certificates
4
5  app = Flask(__name__)
6
7
8  @app.route('/')
9  def homepage():
10     return render_template('home.html')
11
12
13  @app.route('/apply_job')
14  def applyjob():
15     return render_template("apply_job.html")
16
17
18  @app.route('/aboutUs')
19  def aboutUs():
20     return render_template("aboutUs.html")
21
22
23  @app.route('/fill_form')
24  def fillform():
25     return render_template("Form.html")
26
27
28  @app.route(rule='/uploader', methods=['GET', 'POST'])
29  def upload_file():
30     if request.method == 'POST':
31         appliedJob = request.form['job'] # get selected job from the form
32         nameGiven = request.form['name'] # get name from the form
33         emailGiven = request.form['email'] # get email from the form
34         f = request.files['file'] # taking resume file from submitted form
35         f.save(secure_filename(f.filename))
36         name, email, skills = extract_skills(f.filename)
37         skills_matched = compare_skills(appliedJob, skills)
38         # education = extract_education(f.filename)
39         # certificate = extract_certificates(f.filename)
40         # print(education)
41         # print(certificate)
42         print(email)
43         print(skills)
44         print(skills_matched)
45         is_rejected = True
46         if len(skills_matched) >= 4:
47             print("he /she is eligible")
48             is_rejected = False
49             send_email(email, name, is_rejected, appliedJob)
50             return render_template(template_name_or_list='success.html', name=nameGiven, email=emailGiven, skills=skills_matched)
51         else:
52             is_rejected = True
53             print("Sorry, we can't process your candidature")
54
55             send_email(email, name, is_rejected, appliedJob)
56             return render_template(template_name_or_list='success.html', name=nameGiven, email=emailGiven, skills=skills_matched)
57         return render_template('home.html')
58
59
60  @app.route('/home')
61  def homepage2():
62     return render_template('home.html')
63
64
65  if __name__ == '__main__':
66     app.run(debug=False)
67
```

```
* Serving Flask app 'app'
* Debug mode: off
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
```

Output Screenshots

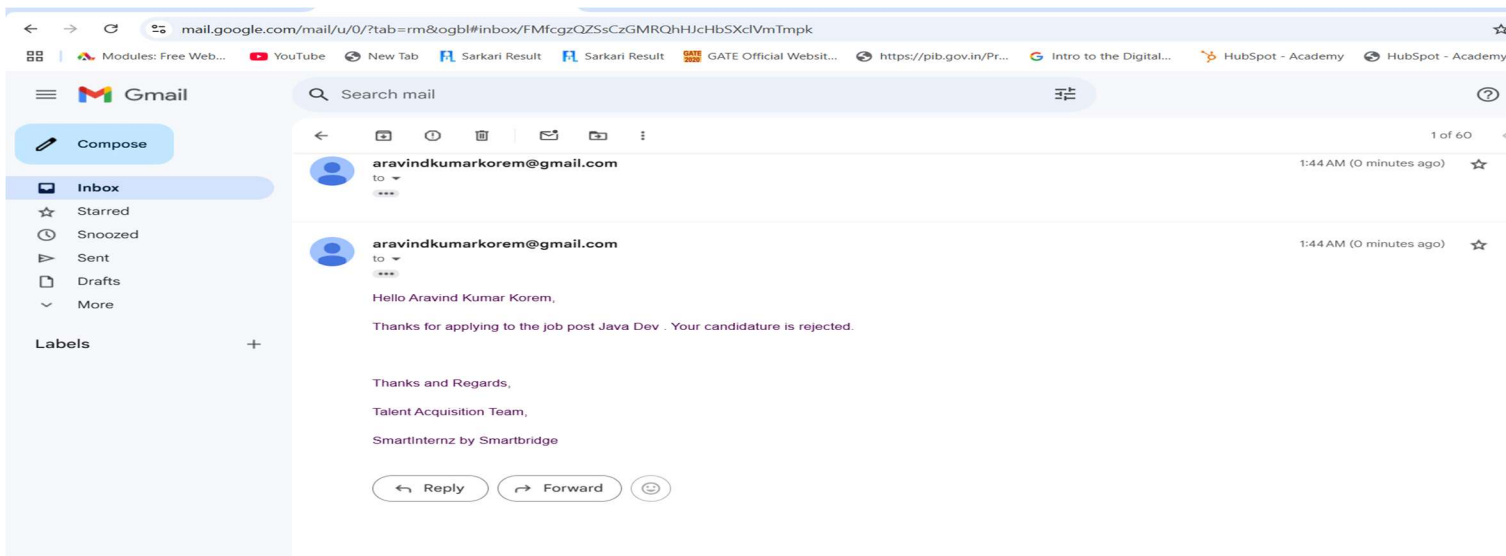
Output1:

Selection Mail:



-Output2:-

Rejection Mail:



10.2 GitHub &Project Demo Link:- Project execution Files [ClickHere](#)

